RAZ EINI

Software Engineering Student • Security, Systems & Backend Engineering

Rishon LeZion, Israel • 053-432-1663 • razeini972@gmail.com • github.com/RazEini • linkedin.com/in/raz-eini

PROFILE

Software Engineering student (Academic Reserve — Atuda "Amirim") targeting cyber and intelligence-unit placement, with hands-on experience across defensive network security, low-level binary exploitation, and applied cryptography. Built a hybrid Python/C Network Intrusion Detection System around a dual-layer Deep Packet Inspection pipeline — an Aho-Corasick automaton paired with a bounds-checked native C extension (ctypes) — measured at a 7.94x throughput acceleration, backed by a fully automated Grafana/Loki/Promtail observability stack. Actively researches memory-corruption vulnerabilities and control-flow hijacking through picoCTF binary-exploitation challenges (GDB+GEF, Ghidra, pwntools). Comfortable operating across the full stack: raw sockets, thread-safe concurrent systems, and cryptographic key derivation / secure vault architecture.

SECURITY & TECHNICAL SKILLS

- **Security & Observability:** Deep Packet Inspection (DPI) — Aho-Corasick multi-pattern automaton & bounds-checked native C signature engine (ctypes); NIDS/NIPS detection logic; Active Threat Mitigation (dynamic OS-level IP isolation — iptables / netsh); Shannon Entropy analysis for DNS-tunneling & covert-channel detection; bitwise TCP-flag analysis (stealth scans — NULL/FIN/XMAS); Grafana + Loki + Promtail (Dashboards-as-Code); Docker Compose.
- **Binary Exploitation & Reverse Engineering:** Stack-based buffer overflows, control-flow hijacking (EIP/RIP overwrite), function-pointer redirection, De Bruijn / cyclic-pattern offset analysis, checksec-driven mitigation analysis (Canary, NX, PIE, RELRO), dynamic debugging (GDB + GEF), static disassembly (Ghidra, objdump), exploit automation (pwntools), 32-bit/64-bit cross-architecture ELF debugging.
- **Networking & Concurrency:** Raw-socket packet capture & analysis (Scapy), OSI/TCP-IP stack (L2–L7), Producer-Consumer pipelines (queue.Queue), thread-safe state management (threading.Lock), bounded-memory backpressure design, background garbage-collection routines.
- **Cryptography & Systems:** Fernet symmetric encryption (AES-128-CBC + HMAC-SHA256), PBKDF2-HMAC-SHA256 key derivation (390K iterations), salting & hashing, C-FFI / native extensions (ctypes), Linux systems & CLI administration.
- **Languages & Infrastructure:** C, Python, Java, Kotlin, Bash; Git & GitHub Actions (CI/CD); Docker; PyPI packaging; pytest.

PROJECTS

WireWarden – Hybrid Python/C NIDS & Observability Engine [View on GitHub ↗]

Python, Native C (ctypes), Scapy, Docker Compose, Grafana, Loki, Promtail

- Architected an end-to-end real-time NIDS/NIPS performing L3–L7 traffic monitoring on a continuous Producer-Consumer pipeline (bounded queue.Queue, maxsize=20000) that fully separates packet capture from analysis, guaranteeing zero packet buffering and predictable memory behavior under sustained load.
- Designed a dual-layer Deep Packet Inspection engine: a pyahocorasick multi-pattern automaton for keyword/credential-leak signatures, paired with a bounds-checked native C extension (memchr/memcmp, loaded live via ctypes) for injection-pattern detection (SQLi, XSS, path traversal, command injection) — measured at a 7.94x throughput acceleration (~2.53M pkts/sec) over pure Python in isolated benchmarking, with zero crash risk on malformed frames.
- Implemented Shannon Entropy analysis ($H(X) = -\Sigma P(x_i)\log_2 P(x_i)$) over DNS query strings to flag exfiltration/tunneling, plus bitwise TCP-flag inspection to catch stealth scans (NULL/FIN/XMAS) and SYN-flood DoS activity via sliding time windows.
- Built Active Defense: dynamic OS-firewall rule injection (netsh / iptables) on threshold breach, executed asynchronously in detached daemon threads to keep the detection pipeline at zero added latency.
- Guarded all shared state (syn_history, port_history, blacklist) with a single threading.Lock for atomic, race-free updates, and added a background garbage-collection thread purging stale records every 30s in bounded O(N) time.
- Shipped a fully provisioned observability stack (Grafana + Loki + Promtail via Docker Compose) with a version-controlled "Dashboards-as-Code" JSON dashboard, auto-loaded on container startup for live security-event visualization.

CTF Writeups – Binary Exploitation & Reverse Engineering [View on GitHub ↗]

C, Python, pwntools, GDB (GEF), Ghidra, Linux (WSL2/Ubuntu)

- Maintain a structured, professionally documented repository of picoCTF binary-exploitation writeups, each following a consistent format: checksec-based mitigation analysis, stack-layout diagrams, root-cause vulnerability analysis, and a working local/remote exploit script.

- Performed control-flow hijacking by overwriting saved return addresses (EIP) on non-PIE, non-canary 32-bit ELF binaries — used De Bruijn (cyclic) pattern generation in GDB/GEF to pinpoint buffer-to-return-address offsets, then redirected execution into an uncalled win() function via little-endian-packed static addresses.
- Exploited function-pointer redirection by resolving symbol addresses directly from unstripped ELF binaries (pwntools ELF()), demonstrating arbitrary code-execution primitives with no memory corruption required.
- Performed targeted, endian-aware single-byte stack-variable overwrites to manipulate program logic (e.g., forcing a comparison to succeed) without corrupting adjacent return addresses or triggering crash-based detection.
- Combined dynamic analysis (GDB+GEF) with static disassembly (Ghidra, objdump) to reconstruct runtime stack state, automating exploit delivery end-to-end with pwntools against both local and live remote targets.
- Solved cross-architecture challenges by configuring 32-bit runtime support (libc6:i386) on a 64-bit Ubuntu/WSL2 host to debug and exploit legacy ELF binaries.

Secure CLI/TUI Password Manager (PyPI Package) View on GitHub ↗

Python, Cryptography, GitHub Actions, PyPI

- Architected a modular, production-grade password vault with a clean src-layout separating cryptographic core logic from CLI/TUI interfaces, published as an installable package on PyPI.
- Implemented authenticated encryption via Fernet (AES-128-CBC + HMAC-SHA256) with PBKDF2-HMAC-SHA256 key derivation at 390,000 iterations, ensuring the master password is never persisted to disk and keys are derived in real time from a per-vault salt.
- Built an arrow-key interactive TUI (questionary + rich) alongside a full CLI (init / add / get / list / delete / change-master / generate / export-csv / import-csv), with clipboard integration (pyperclip) for secure one-keypress retrieval.
- Achieved full unit-test coverage (pytest) across cryptographic and vault-storage logic, and automated the entire release pipeline via GitHub Actions — running tests on every push and publishing production-ready packages directly to PyPI on tagged releases.

EDUCATION

B.Sc. Software Engineering, Afeka College of Engineering | Present | Expected Graduation: 2029

Academic Reserve (Atuda) "Amirim" Program — pre-service technological track

Makif Tet High School | 2022–2025

- High School Diploma (GPA: 91) — Majors in Software Engineering & Cyber (10 units), Physics, Math, and English (5 units).
- Key Achievements: Grade 100 in Systems Design & Programming; Software Technician Certification.

ADDITIONAL

- **Languages:** Hebrew (native), English (high level)
- **Interests:** Chess (Peak 2200 ELO), Speedcubing